

DL for ENG

LECTURE 5.1

Convolutional, Residual and Graph Networks

Look only at the neighbours and reuse the weights: a convolution on a grid, message passing on a graph.

Remus Teodorescu

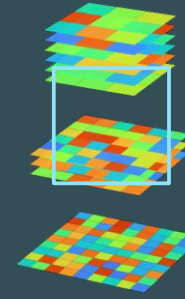
With support from Research Assistant Noman Khan - nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

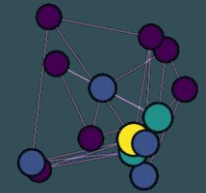
Deep Learning for Engineering · 2026

convolution · residual networks · graph networks · the six-bus network

A GRID AND A GRAPH



a grid — every pixel has the same neighbours



a graph — whoever the edges connect

Left: an image, a grid in which every pixel has the same neighbours. Right: a graph, in which the neighbours are whatever is connected. The same idea in both: look at the neighbours, and reuse the weights.

Learning Objectives

SECTIONS, IN ORDER

- 1 **Local rules** — why a dense layer is a poor fit for an image
- 2 **Convolution** — what a convolutional network is, and its main pieces
- 3 **Residual networks** — skips that let very deep networks train
- 4 **Graphs** — the same idea on data made of connected parts
- 5 **Choosing the architecture** — which one suits a given problem
- 6 **Challenges** — common difficulties, and what to check

WHAT WE SKIP, AND WHERE IT IS TAUGHT

transfer learning reusing a trained network — L6.2

camera network the network on the car — L11.1

physics on a graph physics in a grid network — L12.1

sequences such as time series — L5.2

One small example runs through the lecture, a six-bus power network. You work with it again in the exercises.

Recap

1 – 4 · DEPTH

- 1 What is a deep network, written as a composition, and what does one layer compute in matrix form? *UDL ch. 4.1 – 4.4*
- 2 Why does doubling the width cost four times the parameters while doubling the depth costs two? *UDL ch. 4.4; Ex_04 notebook 04*
- 3 Why do the linear regions multiply rather than add when layers are composed, and what does that say for depth? *UDL ch. 4.5*
- 4 What is the vanishing gradient, and which three changes made deep networks trainable? *UDL ch. 7.5, 11*

5 – 10 · GENERALISATION

- 5 What is model capacity, and how do training and held-out error behave as it grows? *UDL ch. 8.1*
- 6 A model passes through every training point. Why is that not good news, and how would you detect it? *UDL ch. 8.1; Ex_04 notebook 05*
- 7 What are the jobs of the training, validation and test sets, and why is the test set used once? *UDL ch. 8.1*
- 8 What are the three terms in the bias–variance decomposition, and what does an error below the noise floor suggest? *UDL ch. 8.2*
- 9 What does double descent add to the classical U-curve, and when does it matter in practice? *UDL ch. 8.4*
- 10 Given an overfitted model, why regularise before shrinking it, and which two methods would you try first? *UDL ch. 9.1; Ex_04 notebook 05*

Local Rules

A DENSE LAYER ON AN IMAGE, AND WHAT REPLACES IT

- **a 256×256 image** — one input per pixel: $256 \times 256 = 65,536$ inputs
- **a dense layer of 1,000 neurons** — every input to every neuron: $65,536 \times 1,000 \approx 65$ million weights
- **the bigger problem** — to a dense layer the image is a flat list; it cannot see which pixels are neighbours
- **the new rule** — look only at a small neighbourhood, and reuse the same weights everywhere
- **one 3×3 kernel** — 9 weights + 1 bias = 10 parameters, for any image size
- **the lesson** — build in what we know; learning it from data costs parameters and data

ON A GRID — CONVOLUTION

computes	a weighted sum over a small window
parameters	the kernel weights and one bias
reused	at every position, so for any image size

ON A GRAPH — MESSAGE PASSING

computes	a sum over the connected nodes
parameters	one weight matrix and one bias
reused	at every node, so for any graph size

Convolutional Network

KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a feed-forward network whose layers are convolutions
- **function** — image in, features or label out, shift-equivariant
- **architecture** — convolution and pooling, then a dense head
- **applications** — classification, segmentation, detection, 1-D signals

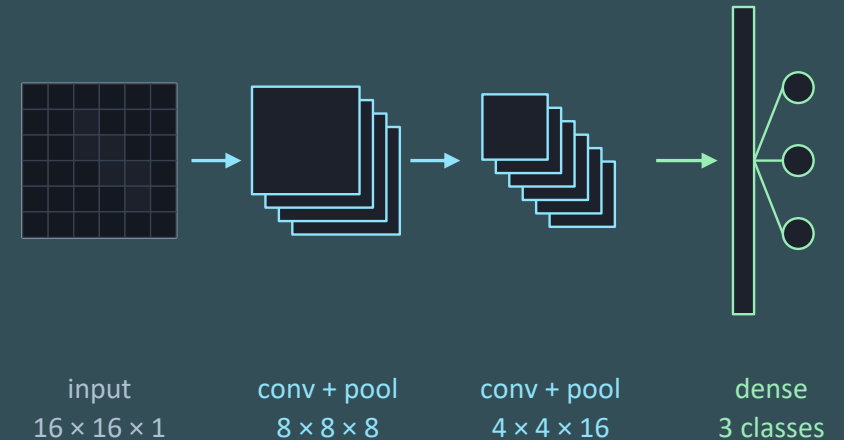
AGAINST A DENSE NETWORK

	dense network	convolutional
connections	every input to every neuron	a small window only
weights	one per connection	one kernel, reused at every position
parameters	grow with the input size	weights and biases, any image size
a shifted input	an unrelated output	the same shift of the output
input size	fixed when designed	any, with global pooling

CONVOLUTION AND NETWORK

$$h_{ij} = a \left(\sum_m \sum_n k_{mn} x_{i+m, j+n} + b \right)$$

x the input image, k_{mn} the kernel, b one bias
 a the activation, h_{ij} one output pixel



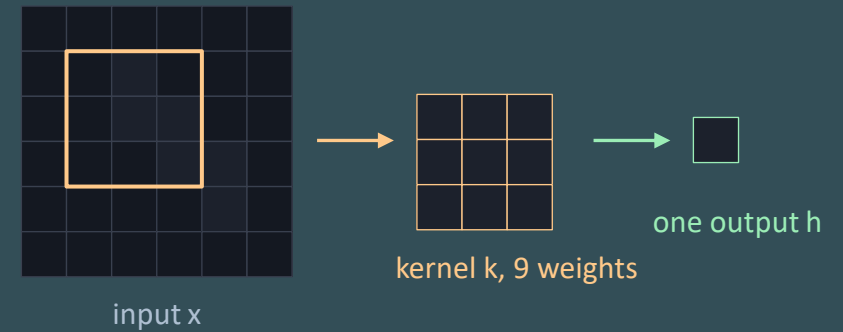
Ex05 notebook 01 · 2,019 parameters · clean, crack, pit

Kernel

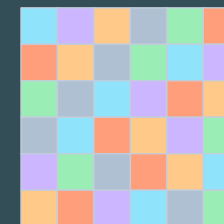
A WEIGHTED SUM OVER A SMALL WINDOW

- a 3×3 window — multiply, sum, add a bias, apply the activation
- move one pixel and repeat — with the same nine weights
- the nine weights are learned — before 2012 an engineer designed them by hand
- the output — an image of the same kind, one value per position
- an edge detector — columns weighted $-1, 0, +1$ respond where brightness changes

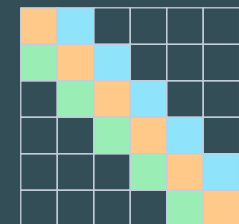
ONE WINDOW, ONE OUTPUT PIXEL



THE SAME IDEA AS A WEIGHT MATRIX



dense
36 weights



convolution
3 weights, reused

Convolution Settings

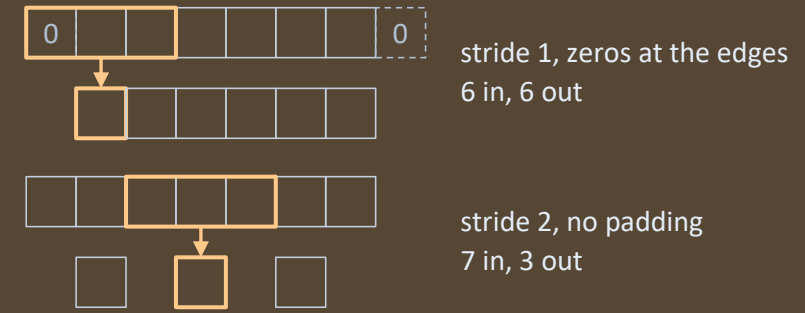
CHOSEN BEFORE TRAINING, THEN LEARNED

- **stride** — the step the window takes; a step of 2 halves the image
- **padding** — a border of zeros, so the output keeps the input's size
- **channels** — features per position; each has its own kernel and bias
- **learned parameters** — kernel area × channels in × channels out, plus one bias per output channel

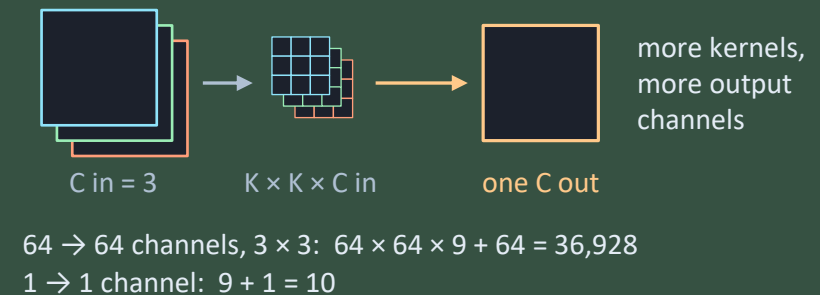
$$P_{\text{dense}} = N_{\text{in}}N_{\text{out}} + N_{\text{out}} \quad P_{\text{conv}} = K^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}$$

$N_{\text{in}}, N_{\text{out}}$ neurons in and out, K the kernel side, $C_{\text{in}}, C_{\text{out}}$ channels in and out

STRIDE AND PADDING, IN 1D



CHANNELS, AND ONE LAYER COUNTED



Pooling and Hierarchy

SHRINK THE IMAGE, BUILD UP FEATURES

- **pooling** — each 2×2 block becomes its largest value
- **why pool** — less computation; small shifts matter less
- **image shrinks, channels grow** — less where, more what
- **the hierarchy** — edges, textures, parts, whole objects
- **nobody designs it** — it emerges from training
- **when not to pool** — when the answer must say where

MAX POOLING, 2×2

1	3	2	0
4	2	1	1
0	1	5	2
2	0	3	4

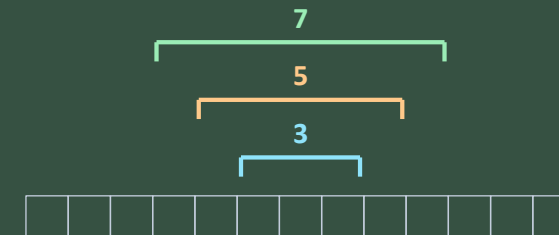
input, 4×4 

4	2
2	5

output, largest of each

WHAT ONE NEURON SEES, BY DEPTH

each layer widens the reach



one row of pixels

layer 3 · parts

layer 2 · textures

layer 1 · edges

Equivariance

EQUIVARIANT AND INVARIANT

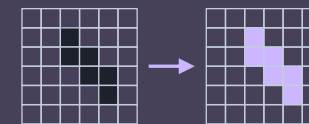
- **equivariant** — move the input, and the output moves with it
- **invariant** — move the input, and the answer stays the same
- **for example** — the crack map moves; “crack: yes” does not
- **convolution** — equivariant: the same kernel everywhere
- **pooling** — adds invariance by throwing position away

TALK · 2 MIN How do you understand convolution — what does it do to an image, in your own words? Then, in the table on the right, which row describes a crack detector that must say where the crack is, and which a classifier that only says whether there is one?

WHAT EACH OPERATION GIVES

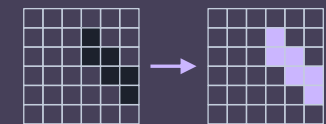
	property	used for
convolution	equivariant	every layer
pooling	partly invariant	classification
global average	invariant	one label per image
no pooling	equivariant	segmentation
message passing	equivariant to renumbering	graphs

input, and its crack map



crack: yes

moved one pixel right



crack: yes

the map moves: equivariant · the answer stays: invariant

Convolution in Practice

WHAT THE CONVOLUTION SECTION SHOWED

- **local and shared** — small window, same weights everywhere
- **few parameters** — kernel weights and biases, for any image size
- **settings** — stride, padding and channels, chosen before training
- **pooling** — shrinks the image; deeper layers see more of it
- **equivariant** — output follows input; pooling adds invariance

IN EX05 NOTEBOOK 01



NOTEBOOK 01, THE RESULT

	parameters	train	held out
convolutional	2,019	0.995	0.983
dense	16,643	0.888	0.722

600 weld radiographs, 16 × 16 · clean, crack, pit

THE SAME RESULT IN UDL

convolutional about 17 % test error

dense about 40 % test error

MNIST-1D · same depth and width · structure built in wins

Residual Network

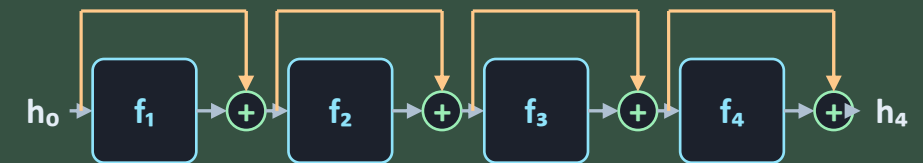
KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — a deep network whose blocks add back their input
- **function** — the same kind of map, but trainable at great depth
- **architecture** — a chain of blocks, each with a skip around it
- **applications** — deep CNNs, graph networks, transformers

AGAINST A DENSE NETWORK

	dense network	residual network
each block	a new representation	a correction to its input
input to output	through every layer	also a direct path, no weights
the gradient	fades with depth	reaches every block
nothing to add	must learn the identity	outputs zero
depth that trains	limited	a hundred layers and more

A RESIDUAL NETWORK, FOUR BLOCKS



amber: the skips, with no weights on them

$$\mathbf{h}_{k+1} = \mathbf{h}_k + f(\mathbf{h}_k, \theta_k)$$

$\mathbf{h}_k$ the input to block k , f the block
 θ_k its parameters, the other $\mathbf{h}_k$ is the skip

each block adds a correction · the skips form a free path

Training Very Deep Networks

WHY PLAIN DEEP NETWORKS FAIL

- **deeper is worse** — without skips, 56 layers train worse than 20
- **not overfitting** — the training error itself is higher
- **vanishing gradient** — a long product of small derivatives
- **shattered gradient** — it changes erratically from input to input
- **the fix** — skips give short paths and a smoother gradient

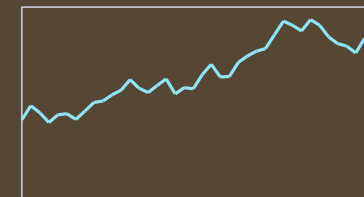
TALK · 2 MIN What does depth give a network that width does not? Then, in the upper plot on the right, the 56-layer network has the higher error on its own training data — why does that rule out overfitting as the cause?

TRAINING ERROR, NO SKIPS



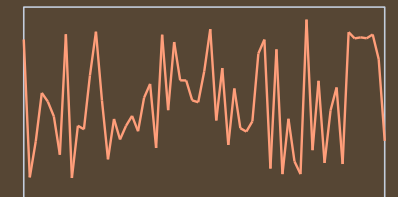
schematic: the shape UDL fig. 11.2 measures

THE GRADIENT, SHALLOW AND DEEP



input, schematic

1 hidden layer: smooth



input, schematic

24 hidden layers: erratic

Normalisation

KEEP EVERY LAYER ON A SIMILAR SCALE

- **the problem** — each skip adds, so the variance doubles per block
- **batch norm** — standardise each channel over the batch
- **then rescale** — a learned scale and offset per channel
- **at inference** — evaluation mode, with averages stored in training
- **layer norm** — standardise each example on its own

$$h_i \leftarrow \gamma \frac{h_i - m_h}{s_h + \epsilon} + \delta$$

m_h , s_h mean and spread over the batch, γ , δ learned scale and offset, ϵ a small number

VARIANCE AFTER EACH BLOCK

skips alone: the variance doubles



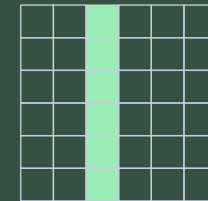
batch norm first in each block: it grows by one



WHICH VALUES SHARE ONE AVERAGE

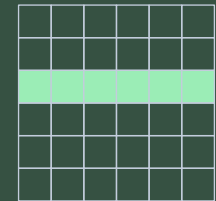
rows:
examples

columns:
channels



batch norm

one channel, all examples



layer norm

one example, all channels

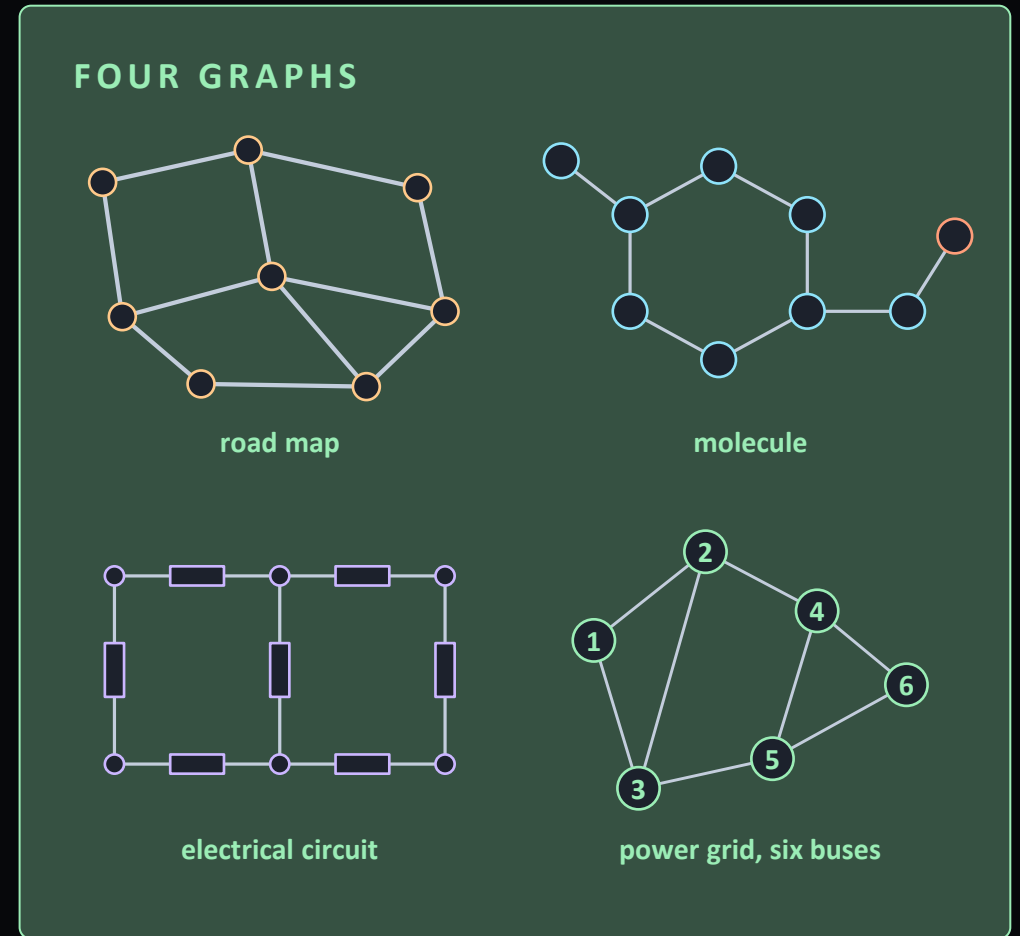
Graphs

NODES AND EDGES, IN FOUR FIELDS

	nodes	edges
road map	junctions	roads between them
molecule	atoms	chemical bonds
electrical circuit	junctions	the components
power grid	buses	transmission lines

THE TYPES WE USE

- **undirected** — a line carries power both ways
- **directed** — one way only, like one paper citing another
- **edge features** — numbers on each edge, like line admittance



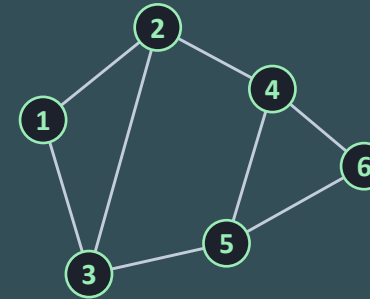
Graph Representation

THREE MATRICES DESCRIBE THE WHOLE GRAPH

- **node features X** — one row per node: what is measured there
- **adjacency A** — a one wherever two nodes are connected
- **edge features E** — one row per edge: here, a line's susceptance
- **sparse** — 16 non-zeros of 36; real grids are far sparser

TALK · 2 MIN How would you explain a graph to someone outside engineering, and what makes a power grid one? Then, in the adjacency matrix on the right, which row belongs to bus 2, and which three columns of it are lit?

THE SIX BUSES AS MATRICES



	1	2	3	4	5	6
1						
2						
3						
4						
5						
6						

A, 6 × 6

P	Q	g	l	r

X, 6 × 5

1-2	4
1-3	3
2-3	3.5
2-4	2.5
3-5	3
4-5	3.5
4-6	2
5-6	2.5

E, 8 × 1

Graph Neural Network

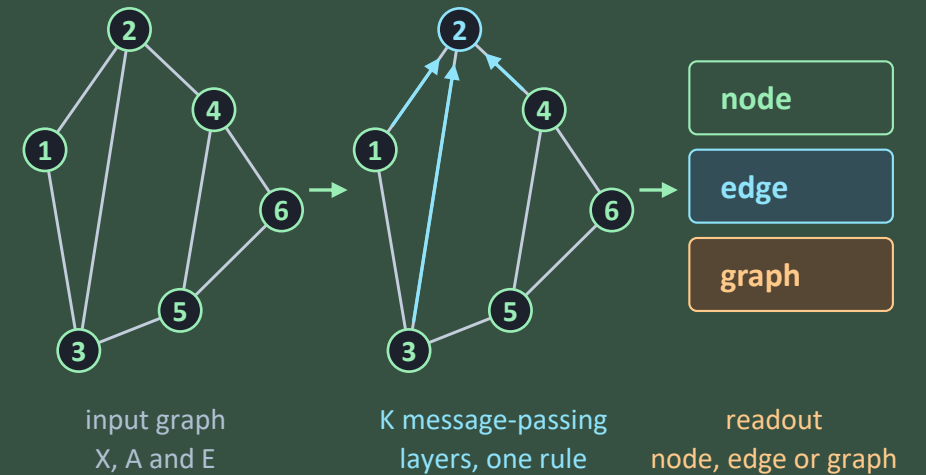
KIND, FUNCTION, ARCHITECTURE, APPLICATIONS

- **kind** — layers that update every node from its neighbours
- **function** — graph in, an answer per node, edge or graph out
- **architecture** — K message-passing layers, then a readout
- **applications** — power grids, circuits, molecules, road maps

AGAINST A DENSE NETWORK

	dense network	graph network
input	one vector, fixed length	a graph of any size, with features
connections	every input to every neuron	along the edges only
weights	one per connection	one rule, shared by every node
renumbered nodes	an unrelated output	the same output, renumbered
topology	fixed when trained	an input; one model, many graphs

THE ARCHITECTURE, ON THE SIX BUSES



bus 2 gathers from buses 1, 3 and 4 · every bus does the same, with the same rule

Graph Tasks and Losses

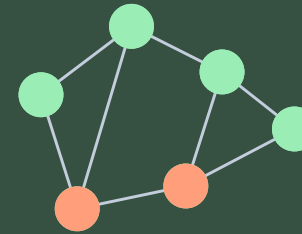
WHAT IS PREDICTED, AND HOW IT IS SCORED

task	output	loss
node regression	a number per node	squared error, as in notebook 03
node classification	a class per node	cross-entropy
edge prediction	a probability per edge	binary cross-entropy
graph classification	one class per graph	cross-entropy, on the averaged nodes

A NEW GRAPH, NO RETRAINING

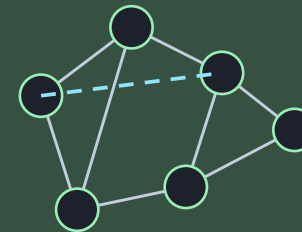
- **inductive** — train on some graphs, predict on new ones
- **a tripped line** — a new graph; notebook 03 predicts it directly
- **transductive** — one large graph, predict its unlabelled nodes

THREE LEVELS OF ANSWER



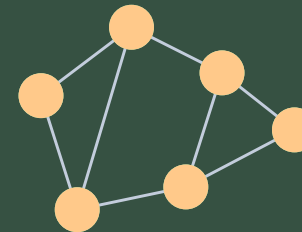
node

a value at every bus, as in notebook 03



edge

a probability for each line, such as whether it is present



graph

one answer for the whole grid, such as whether it is stable

Message Passing

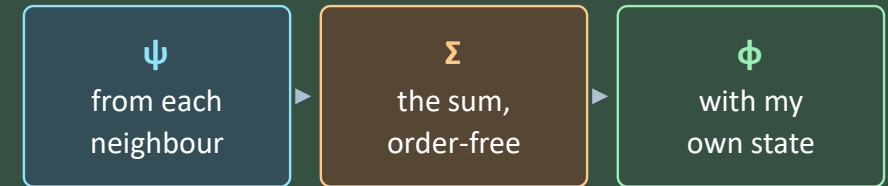
THREE STEPS, REPEATED AT EVERY NODE

- **for each node** — compute a message from each neighbour
- **sum the messages** — then combine with the node's own state
- **both functions are small networks** — shared across every node
- **the sum** — ignores order and count: one model, any graph size
- **K layers reach K hops** — match K to the graph diameter

$$\mathbf{h}_{i^{(l+1)}} = \phi \left(\mathbf{h}_{i^{(l)}}, \sum_{j \in \mathcal{N}(i)} \psi(\mathbf{h}_{i^{(l)}}, \mathbf{h}_{j^{(l)}}) \right)$$

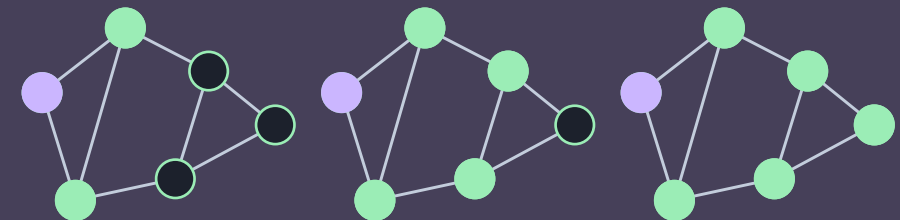
$\mathbf{h}_i^{(l)}$ node i at layer l , $\mathcal{N}(i)$ its neighbours, ψ the message, ϕ the combine

ONE NODE'S UPDATE



same functions at every node · repeat for more hops

HOW FAR BUS 1 HEARS, LAYER BY LAYER



layer 1: 3 of 6 buses

layer 2: 5 of 6 buses

layer 3: 6 of 6 buses

diameter 3: three layers reach every bus

Graph Convolution

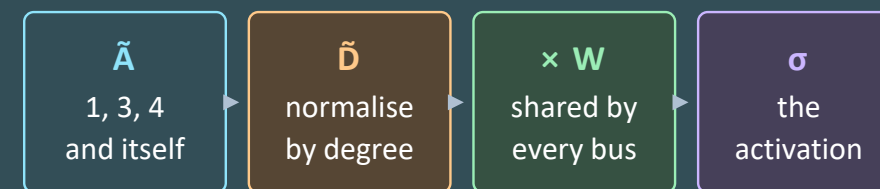
THE SIMPLEST CHOICE, IN ONE MATRIX LINE

- **the message** — the neighbour's state, unchanged
- **the combine step** — a shared linear map, then an activation
- **read right to left** — mix features, sum over neighbours, activate
- **add the identity** — so a node's own state is included
- **normalise by degree** — or sums grow with the neighbour count

$$\mathbf{H}^{(l+1)} = \sigma(\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H}^{(l)} \mathbf{W}^{(l)})$$

$\mathbf{H}^{(l)}$ all node features, $\mathbf{W}^{(l)}$ shared weights, $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, $\tilde{\mathbf{D}}$ degree, σ the activation

ONE LAYER, AT BUS 2



every bus, the same W · one layer is one matrix line

THE SAME SHAPE AS A CONVOLUTION

	convolution	graph convolution
neighbourhood	a 3×3 window	neighbours and itself
weights	one kernel	one matrix W
shared over	every position	every node

Permutation Equivariance

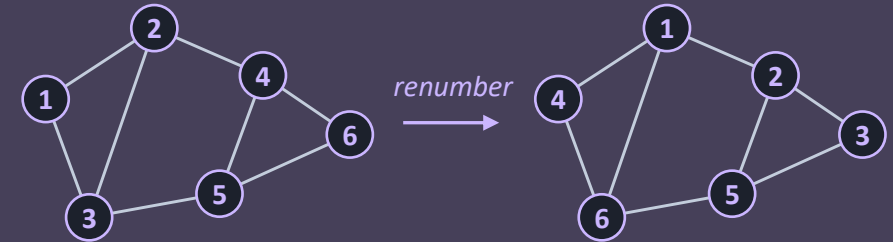
RENUMBER THE NODES, THE OUTPUT FOLLOWS

- **numbering the buses is bookkeeping** — there is no natural order
- **an answer that changes with the numbering** — is wrong, not just inaccurate
- **a graph network** — renumber the input and the output follows exactly
- **a dense network** — fails this, and notebook 03 measures by how much

$$f(\mathbf{PX}, \mathbf{PA}\mathbf{P}^T) = \mathbf{P} f(\mathbf{X}, \mathbf{A})$$

$\mathbf{X}$ node features, $\mathbf{A}$ the adjacency, $\mathbf{P}$ any relabelling of the nodes, f the network

ONE GRID, TWO NUMBERINGS



same lines, same physics · only the labels changed

MEASURED IN EX05 NOTEBOOK 03

graph network	gap 1.7×10^{-16} rad: none
dense network	gap 0.57 rad
dense, relabelled	153 × worse

Choosing the Architecture

START FROM THE SHAPE OF THE DATA

a fixed list of numbers	dense network
a grid: image, signal	convolutional network
a graph: grid, circuit	graph network
a sequence in time	next lecture
many layers, any kind	add residual connections

MEASURED IN NOTEBOOK 03, ANGLE ERROR

	intact grid	a line tripped
dense network	0.0016 rad	0.265 rad
graph network	0.0043 rad	0.115 rad

dense wins on a fixed grid · after a trip, neither is usable

WHY THE GRAPH NETWORK LOSES HERE

six nodes	a dense network learns every relation
fixed topology	the structure is known exactly
physics known	the admittance matrix holds it

WHEN IT EARNS ITS PLACE

topology changes	one model for many grids
large graphs	the cost grows with the edges
renumbering	the answer must not change

Choose by what the architecture buys in your problem, not by fashion.

Challenges

A MISSING VALUE IS NOT ZERO

a missing measurement read as zero looks like a real zero · add a mask channel that says which values exist

OVER-SMOOTHING

too many graph layers make every node alike · keep the depth near the graph diameter · add residual connections

FIXED INPUT SIZE

a dense head expects one image size · use global average pooling, or resize the input

SIX CHECKS BEFORE TRUSTING A MODEL

print	every tensor shape once
check	the adjacency is symmetric
permute	the nodes — does the output follow?
shrink	the depth to the diameter
mask	a channel for missing values
test	on a changed topology

the permutation test: four lines of code · it shows at once whether the model has the property it was built for

Exercise 5

NOTEBOOKS 01 TO 03, AND 04 AFTER L5.2

- **01** — a small CNN on weld radiographs: clean, crack, pit
- **02** — message passing by hand in NumPy, before any library
- **03** — node regression on the six buses, with the permutation test
- **04** — a sequence task, after the next lecture



THE QUESTION EACH NOTEBOOK ANSWERS

- 01** does the CNN beat a dense network with far fewer parameters?
- 02** why does summing over neighbours make order and count irrelevant?
- 03** does the output follow a renumbering, and how badly does dense fail?

REPORT QUESTION

The dense model was more accurate. Would you still use the graph network, and on what evidence?

Key Takeaways

1 What kind of network

Convolutional: layers are convolutions. Residual: blocks add back their input. Graph: nodes updated from their neighbours.

2 What function it implements

A grid in, features or a label out, shift-equivariant. The same map, trainable at depth. A graph in, an answer per node, edge or graph.

3 What the architecture is

Convolution and pooling, then a dense head. A chain of blocks with skips. K message-passing layers, then a readout.

4 How it differs from a dense network

Local connections and shared weights, so few parameters and a symmetry built in; the skips give the gradient a free path.

5 What it is used for

Images and signals. Any network that must be deep. Power grids, circuits, molecules. On six fixed buses, dense still wins.

Next — L5.2: RNN, LSTM, attention and transformers.

Questions

TEN QUESTIONS FOR THE ORAL EXAMINATION

- 1 What kind of network is a convolutional network, and what function does it implement? *UDL ch. 10.1 – 10.3*
- 2 Describe the architecture of a convolutional network, layer by layer: what happens to resolution and channels? *UDL ch. 10.3 – 10.4; Ex_05 notebook 01*
- 3 How does a convolutional network differ from a dense network in connections, weights, parameters and symmetry? *UDL ch. 10.2; Ex_05 notebook 01*
- 4 Given those differences, what are convolutional networks used for, and which task needs equivariance, not invariance? *UDL ch. 10.1, 10.5*
- 5 Why does a residual connection help the gradient, and what does a block with nothing to add learn? *UDL ch. 11.1 – 11.2*
- 6 What kind of network is a graph network, and what function does it implement on node features and adjacency? *UDL ch. 13.1 – 13.3*
- 7 Describe the architecture of a graph network: the three steps of a layer, the readout, and K against the diameter? *UDL ch. 13.4, 13.7; Ex_05 notebook 02*
- 8 How does a graph network differ from a dense network, and what does permutation equivariance guarantee? *UDL ch. 13.2; Ex_05 notebook 03*
- 9 Given those differences, what are graph networks used for, and what is the difference between inductive and transductive? *UDL ch. 13.3, 13.6*
- 10 When is a graph network the wrong choice, and what did notebook 03 measure that shows it? *Ex_05 notebook 03*

NEXT

L5.2 — RNN, LSTM, Attention and Transformers

Today's structures were spatial, a grid and a graph. Next time the structure is temporal: how to carry information from the start of a long sequence to its end.

BEFORE THE NEXT SESSION

read	UDL ch. 12.1 – 12.3
run	Ex_05 notebooks 00 to 03
bring	your permutation-test numbers
think about	layer count against graph diameter
<i>bring your renumbering-test numbers from notebook 03</i>	